

EX1-S-Cache Modelling

COMP25212 System Architecture

Aim

To investigate cache memory performance through simulation, and use models to explore system performance for a range of cache organisations.

Learning Outcomes

ILO2 Compare the strengths and weaknesses of different system architectures.

ILO3 Explain how and predict to which extent such techniques impact the performance, reliability and flexibility of computing systems.

Deliverables

Coursework submission is via Blackboard (see *Coursework Exercises* page), and includes the following:

- Code implementing the different cache organisations
- Results from modelling the performance of different caches
- Analysis and discussion of cache performance

Introduction to Cache Memory

A cache is literally a ‘secret hiding place’. Cache techniques, both hardware and software, are used extensively in computer systems. This assignment concerns cache memory which is always implemented in hardware. Cache memory is a small amount of very fast memory used as a temporary store for frequently used instruction and data memory locations. It is used because access time for the main RAM memory is slow compared with instruction execution speeds. Therefore avoiding main memory accesses wherever possible, both for instruction fetching and for data accessing, improves overall performance.

Cache memory relies on the fact that in most well behaved large programs, only a small proportion of the total code and data are accessed during any short time interval. If we could keep in cache memory those fragments of code and data that are currently being used, then we could improve performance. A ‘unified’ cache is one used for both instructions and data. We will also consider later the use of separate caches for instructions and data. Under some circumstances two separate caches can perform better than a single unified cache.

On each memory read (including instruction fetches) the required item is first looked for in the cache. This requires a comparison between the desired memory address and the addresses of items in the cache. If the required address is found this is a cache ‘hit’. The item is accessed from the cache and no main memory access is necessary. Only if the item is not present in the cache is main memory accessed. When main memory is accessed the item is copied into the cache, in anticipation of possible future accesses.

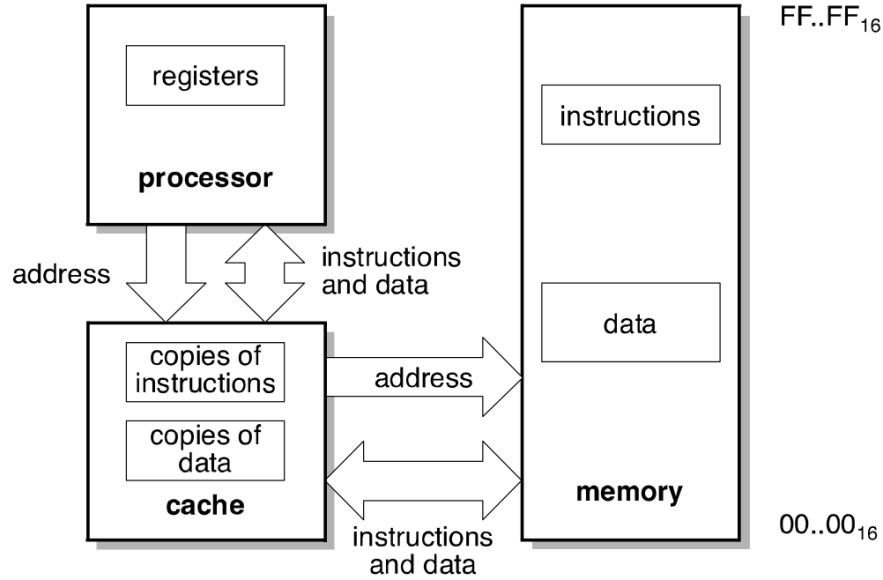


Figure 1: Unified cache containing data and instructions (all figures reproduced with permission from [1])

This in turn may require the rejection of some existing item in the cache to make space for the new item.

Each entry in the cache, comprising data and addressing information, is known as a cache line. In order to compare the desired memory address and the addresses of items in the cache it is necessary for each cache line to have typically three fields. The first field is address information, sometimes called a ‘tag’ field which enables us to find the correct data. The second field is the ‘data’ field which, depending on the cache line size, may contain several words of data including the data item actually required. The third field is a Boolean ‘valid’ bit to indicate whether the line contains currently valid data.

There are several ways of organising and using these cache fields. We consider fully associative, direct mapped and set associative organisations within this introduction.

Fully Associative Cache

In the fully associative model, the tag field is a full byte address (32 bits in our case) truncated to a multiple of a power of two depending on the cache line size. So with a line size of 4 bytes, the bottom 2 bits can be discarded, with 8 bytes the bottom 3 bits etc. Any data can occupy any line of the cache. In order to find whether an item is in the cache, the presented address, suitably truncated, is compared with the tag field of each valid line. In hardware, speed is achieved by comparing all tag fields simultaneously in parallel using associative memory (sometimes called content addressed memory or CAM). If the required address is found this is a cache ‘hit’. If the item is an instruction to be fetched or data to be read, the required value is read from the data field of the cache and the operation is complete.

If there is a cache miss then the data is read from main memory. Enough data to fill the complete cache line is copied into the cache, in anticipation of possible future accesses, and the corresponding tag and valid fields updated.

In a fully associative cache any item may occupy any line so a line must be chosen for the new item.

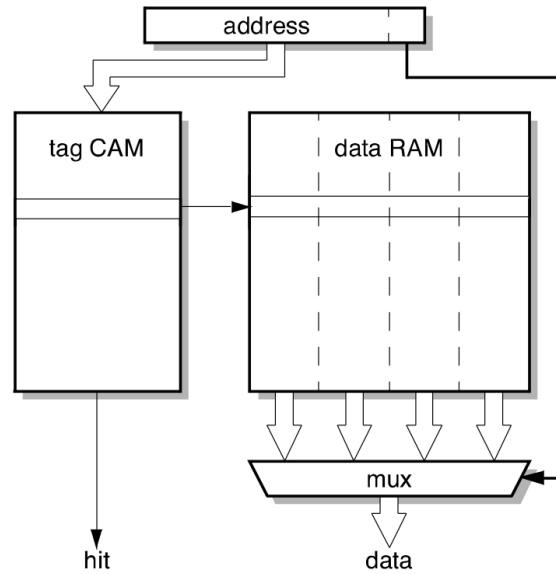


Figure 2: Fully associative cache organisation

If the chosen line is already valid the item already in the cache must be rejected to make space for the new item. Various rejection algorithms can be used, we will use a simple cyclic algorithm where each cache line is rejected in turn.

If the item is to be written the situation is more complicated. The basic technique we will use is to first establish whether the address to be written is in the cache. If it is then the item can be written to the cache. If it is not it will not be written to the cache. In both cases it will be written to main memory so that the memory is up to date as soon as the write takes place, regardless of whether the written item is cached. This strategy, called “write through with no allocate”, is one of several possible writing strategies. In this strategy the write to memory is usually buffered so that it incurs no cost, except when memory contention occurs. If the item is subsequently read, it will be cached at that point.

Direct Mapped Cache

In a direct mapped cache the address is truncated as before, depending on the cache line size. The truncated address is then divided into two parts: a tag, and an index. The index is used to address a cache line directly, and the rest is the tag which is stored along with the data. The size of the index depends on the number of lines in the cache, which must be a power of two. For example, a 32 line cache would use 5 address bits to select a cache line, and the remaining bits would be the tag.

When a new address is presented to the cache, the index is formed and used to address the cache. However, the selected line may contain one of many tag values and so the stored tag must be compared with the tag formed from the presented address. If they are the same then the stored entry must have the same overall address and this is thus a cache hit, otherwise it is a miss. On a miss there is no choice about which cache line must be used, the data must be loaded into the selected line and the tag field updated. The technique on write is similar to that for the fully associative cache.

A direct mapped cache is cheaper to implement than a fully associative cache. It is really just a hash

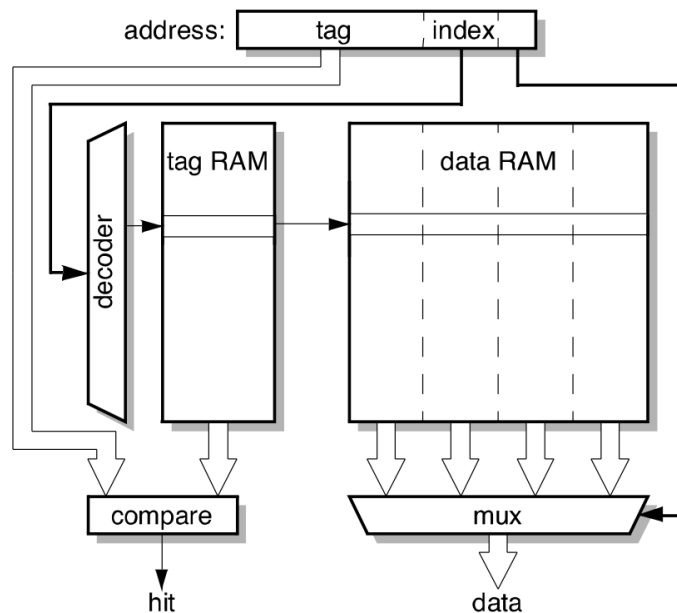


Figure 3: Direct-mapped cache organisation

table implemented in hardware. The disadvantage is the probability of a lower proportion of ‘hits’ to total accesses (the hit rate) because of the inflexible replacement strategy.

Set Associative Cache

A widely implemented compromise between the cost of an associative cache and the disadvantage of a direct mapped cache is a set associative cache. This is a combination of the two previous techniques. The set associative cache is implemented as a small number of direct mapped caches operating in parallel. An n -way set associative cache (where n might be typically 2 or 4) uses n direct mapped caches. An item might be placed in the relevant line of any of the direct mapped caches. So finding an item involves looking at the relevant line in all the caches and doing an associative search on the tag fields to find which is the correct one.

Set associative caches are covered in the lectures but will not be investigated in this practical exercise.

The Problem

The objective of this exercise is to write a simulation in Java, modelling the operation of cache memory. This will then be used to compare the performance of various cache organisations, in particular the hit rates achieved.

A stream of addresses will be fed to your simulator from a trace file, which was recorded by monitoring memory accesses made by a real program. Each of the addresses in the trace file is accompanied by information describing the type of access being made: data read, data write or instruction fetch. Only memory accesses are to be simulated; all other aspects of the behaviour of the program which produced the trace will be ignored. So the simulation will maintain the tag fields in the caches correctly. However the actual data read and written is not to be considered, with no real data being written to the data field of the caches. Main memory will not be simulated in any way, therefore any mention of notional

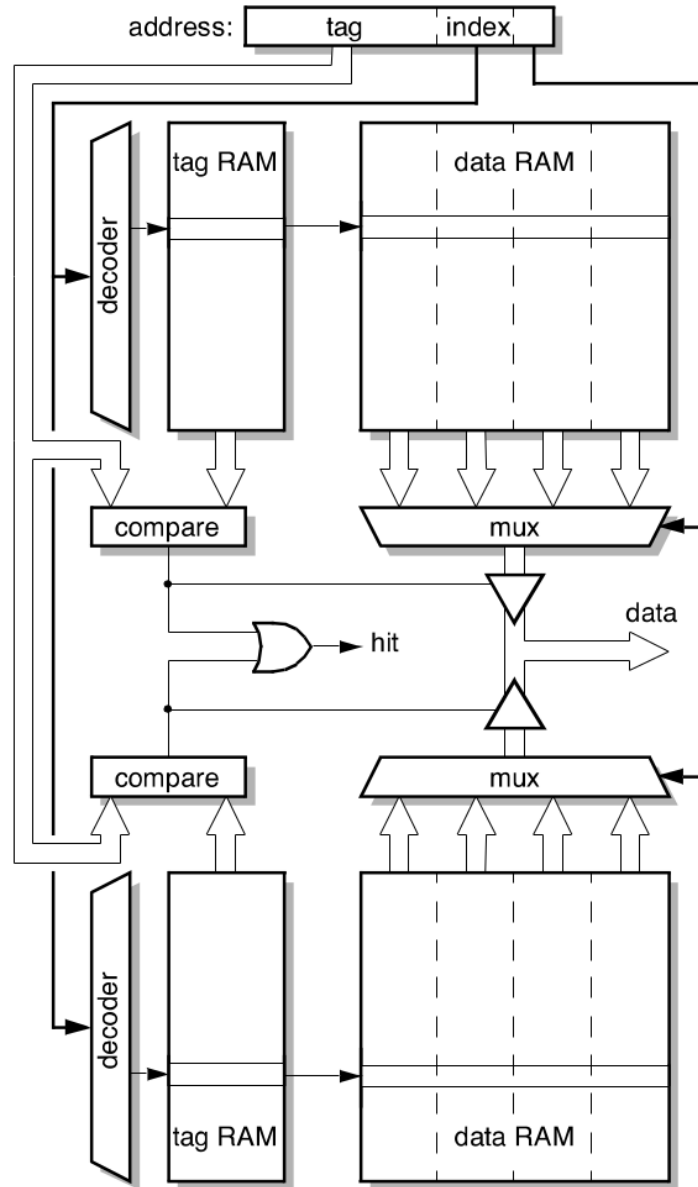


Figure 4: Two-way set associative cache organisation

memory accesses below can just refer to a dummy data value.

An example software framework on which to structure the problem is provided, which facilitates testing of the cache simulators. The problem will therefore be tackled as follows:

- 1 Understand the given code and run it on simple data in conjunction with the ‘Secret’ cache example provided.
- 2 Implement a Direct Mapped cache and test it on simple data to verify correctness, then profile on real data to evaluate performance.
- 3 Implement a Fully Associative cache and test it on simple data to verify correctness, then profile on real data to evaluate performance.

- 4 Compare and analyse the comparative performance of the implemented caches with various cache configurations.

The implementation will allow the total cache size and the cache line size to be any power of 2, allowing the performance of different cache sizes to be compared. It will also allow the possibility of a single unified cache, containing both program instructions and data, or separate data and instruction caches.

Two trace files are provided to be used as follows:

- `traceA`: a simple artificial file intended for testing your code.
- `traceB`: a real trace file of 40000 entries which should give meaningful results and which will be used for performance comparisons.

1 Preparation: Understand the provided code

The file `Cache.java` contains the definition of an abstract class `Cache` which has 3 abstract methods: **`cacheSearch`**, **`cacheNewEntry`** and **`cacheWriteData`**. In this exercise, you will write a class which first implements **`DirectMappedCache`** and, in the next section, **`FullyAssocCache`**. For each of these you will need to define appropriate internal data structures, write a constructor to initialise these structures, and design and code the bodies of the methods.

`Cache.java` also has a number of variables intended for collection of statistics, a method **`dumpStats`** to print these statistics, and a number of methods to return individual statistics if required. In the file `CacheSim.java` you are provided with a skeleton main method which opens the trace file provided as the first argument of the program run as:

```
$ java CacheSim tracefile
```

The skeleton main method then calls the constructor to create a cache (initially a “secret” cache with a total size of 1024 bytes and a line size of 4 bytes) and calls the **`doSimul`** method using the created cache as a unified cache for both data and instructions. **`doSimul`** calls **`getAccess`** to return the next **`MemAccess`**, comprising an **`atype`** (Read, Write or Fetch) and an **`addr`**, from the trace file. If the **`atype`** is a Read or Write the data cache is accessed, if it’s a Fetch, the instruction cache is accessed. **`doSimul`** is written so that it will work equally well with a unified cache or with separate data and instruction caches. At the end of the trace file **`MemAccess`** returns a null pointer which is caught in **`doSimul`** to end the simulation run and call **`dumpStats`** to print the accumulated statistics. You do not need to understand the detailed working of **`getAccess`**.

The major functionality of the **`Cache`** class is provided by the methods **`read`** and **`write`** which access the cache. These are common for all cache types but call the methods **`cacheSearch`**, **`cacheNewEntry`** and **`cacheWriteData`** which perform the detailed operations for a particular cache type. Each cache type should therefore be a class which extends the **`Cache`** class and implements the above abstract methods. The following descriptions of **`read`** and **`write`** are given to aid understanding.

`public Object read (int addr)` Calls **`cacheSearch`** to find if the item addressed by **`addr`** is in the cache. If it is, then the entry is returned. Otherwise **`cacheSearch`** returns null and **`read`** calls **`cacheNewEntry`** to allocate the cache entry (notionally reading main memory in the process). Cache statistics are updated appropriately according to the results returned by **`cacheSearch`** and **`cacheNewEntry`**. The data **`Object`** accessed is returned although this does not contain any valid value and is not actually used. As **`addr`** is a Java int you may assume throughout that all addresses are 32 bits.

public void write (int addr, Object indata) Calls **cacheSearch** to find if the address, **addr**, is in the cache. If it is, **cacheWriteData** is called to update the cache data, otherwise the cache is left unchanged (implementing write through with no allocate). The statistics are updated appropriately. For simplicity, it is assumed that the write through is buffered and that there is no memory contention so writing to memory does not incur any cost. Hence writes are not counted in the total of memory accesses. The data written is the **Object indata** but, as this is not actually used anywhere, the calls to write within **doSimul** use a dummy **Integer(0)** object.

Make sure that you understand fully the rest of the given code. Here is a brief specification of the 3 methods for which you have not been given any source code (note that these 3 methods do not update any cache statistics):

public Object cacheSearch(int addr) Identifies whether the item at address **addr** is in the cache, returning null if not. Otherwise returns an **Object** set to the value of the notional data. If a valid entry is found, its position should be remembered for use by **cacheWriteData** if it is called subsequently by a write operation.

public oldCacheLineInfo cacheNewEntry(int addr) Allocates the new entry to the appropriate cache line for the presented address **addr**. Notes the existing value of the valid bit in the **old.valid** field of the result **oldCacheLineInfo**. Sets all 3 fields of the cache line, notionally accessing main memory to get the data value. Returns the old data value in the data field of the result so that it can be notionally written to main memory if required. The read method notes rejections in the statistics but the old data can be ignored as we are not concerned with actual data values.

public void cacheWriteData (Object data) Simply updates the data field of the cache line using the data **Object** supplied as parameter. It assumes the cache line to be the same line that was found during the last search.

You are provided with the precompiled `SecretCache.class` which extends `Cache.java` and implements **cacheSearch**, **cacheNewEntry** and **cacheWriteData** for an undisclosed cache type. This is provided so that you can observe how the program should behave when given a valid cache implementation. The exact statistics will however vary with the cache type.

Running the Secret Cache Simulation

Compile the supplied `CacheSim.java` program, remembering to leave the `SecretCache.class` file untouched. Run the code against the file `traceA` using the command:

```
$ java CacheSim traceA
```

The code first uses a cache of size 32 bytes with a line size of 8 bytes, then a cache size of 8192 bytes with a line size of 32 bytes. When executed correctly the results should match those detailed in Tab. 1.

2 Implementation of a Direct Mapped Cache

The objective is to model a direct mapped cache, through design, implementation and testing of the class **DirectMappedCache**. To do this you will need to define appropriate internal data structures, write a constructor to initialise these structures and design and code the bodies of the three methods:

Cache size	32	8192
Line size	8	32
Total accesses	16	16
Total read accesses	14	14
Total write accesses	2	2
Read misses	7	3
Write misses	1	1
Total rejections	3	1
Total main memory accesses	7	3
Read miss rate	50.0%	21.4%
Write miss rate	50.0%	50.0%

Table 1: Output from unified **SecretCache** on `traceA`

cacheSearch, **cacheNewEntry** and **cacheWriteData**.

As discussed in the introduction, a tag field and a valid field are required for each cache line, along with storage for the data. It is suggested to define each of these as an array which is created and initialised by the constructor. The data field is not strictly needed for these simulations, but for completeness and generality it might be represented as an array of **Objects**.

For the direct mapped cache, the useful part of the presented address is divided into 2 parts as shown in the table below. The least significant bits would be used to select a byte within a cache line and play no part until the actual data is accessed which is not part of our simulation. These bits can therefore be ignored. The number of bits to be ignored depends on the number of bytes in each cache line. The cache line is selected using the index field which we choose to be the least significant non-ignored bits. The number of bits in the index field depends on the number of cache lines. Once the correct cache line

tag	index	ignored
-----	-------	---------

Table 2: Division of (32-bit) address for direct mapped cache

is selected, the cache tag field can be compared with the tag which is the remaining most significant bits of the presented address. In Java, the fields are most conveniently selected using suitable integer division and modulo operations (although you could also use log, exp or shift operations). Note that the size of the cache in bytes and the size of each line in bytes are supplied as a variable parameters. They should be supplied to the constructor which can then deduce the number of lines. It is a mistake to build in fixed numbers.

The **cacheSearch** method can now be implemented to check whether the required data is present, and return null if not. It can be assumed that **cacheSearch** will always be called before any call to **cacheNewEntry** or **cacheWriteData**, and that **cacheSearch** notes which line is to be used if the latter is called. The **cacheNewEntry** method returns the previous value of the valid bit as well as updating the cache line. **cacheWriteData** merely updates the data field of the noted line.

Modify the main method in `CacheSim.java` to create and test a unified **DirectMappedCache** with the same cache and line sizes as in the Secret Cache example. Test your class using `CacheSim.java` and the trace file `traceA`, which should yield the results detailed in Tab. 3.

Cache Size	32	8192
Line Size	8	32
Total accesses	16	16
Total read accesses	14	14
Total write accesses	2	2
Read Misses	9	3
Write Misses	2	1
Total rejections	5	0
Total main memory accesses	9	3
Read Miss Rate	64.29%	21.4%
Write Miss Rate	100.0%	50.0%

Table 3: Output of unified **DirectMappedCache** on `traceA`

Submission

Once you are satisfied with performance on `traceA`, you should run the simulation with `traceB` which represents a large number of real-world memory accesses. You should record and submit the output from `traceB` to Blackboard, as the recorded values form part of the assessment. You should also upload the code file `DirectMappedCache.java`, which will be used to assess your implementation of the cache functionality. For further details, please check the assignment submission under the *Coursework Exercises* page on Blackboard.

3 Implementation of a Fully-Associative Cache

The next part of this exercise is to implement the class **FullyAssocCache** as a fully associative cache. This should be done without changing `Cache.java` which must continue to work with any type of cache. The constructor performs initialisation similar to that for the direct mapped cache, and as before it is suggested to implement the internal data structures using arrays. The tag field will now comprise all the address bits except the ignored bits.

tag	ignored
-----	---------

Table 4: Division of address for fully associative cache

The method **cacheSearch** must now look at the tag and valid bits of every line of the cache until a match is found. If none is found null is returned. Otherwise the data is returned and the position in the cache noted. In a real associative cache the search of every line is done in parallel by hardware for speed, however in this software simulation, a linear search is appropriate.

The method **cacheNewEntry** will put the given entry into the next line given by the cyclic rejection and allocation algorithm. This implies that an index must cycle through the cache entries. This index should be initialised in the constructor and updated on each call to **cacheNewEntry**.

The method **cacheWriteData** should update the data field of the cache line previously noted by **cacheSearch**. Again you should modify the main method in **CacheSim** to create and test a unified `FullyAssocCache.java` using the same cache and line sizes as previously. When executed with `traceA`, the simulation should produce the results detailed in Tab. 5.

Cache Size	32	8192
Line Size	8	32
Total accesses	16	16
Total read accesses	14	14
Total write accesses	2	2
Read Misses	7	3
Write Misses	1	1
Total rejections	3	0
Total main memory accesses	7	3
Read Miss Rate	50.0%	21.4%
Write Miss Rate	50.0%	50.0%

Table 5: Output of unified **FullyAssocCache** on `traceA`

Submission

Once you are satisfied with performance on `traceA`, you should run the simulation with `traceB` which represents a large number of real-world memory accesses. You should record and submit the output from `traceB` to Blackboard, as the recorded values form part of the assessment. You should also upload the code file `FullyAssocCache.java`, which will be used to assess your implementation of the cache functionality. For further details, please check the assignment submission under the *Coursework Exercises* page on Blackboard.

4 Analysis

The parameterised cache models can now be used to investigate the effect of cache size and organisation on performance, for example exploring the effect of cache line size, or the impact of separate instruction and data caches. The coursework submission on Blackboard contains a series of questions which should be answered using cache simulations and associated theory – please see the *Coursework Exercises* page on Blackboard for further details.

References

- [1] Steve Furber. *ARM System Architecture*. Addison Wesley, 2nd edition, 2000.